Retail Mind AI: Intelligent Product Knowledge Engine

Project Description:

1. Project Overview:

This system is an AI-powered product knowledge assistant designed to provide accurate, context-aware answers to customer support queries, including warranty details, return policies, refund timelines, product compatibility, and order-related issues. It enables users and support agents to interact using natural language, ensuring that responses are relevant, precise, and derived from a structured support knowledge base.

The platform implements a retrieval-based architecture enhanced with semantic search using vector embeddings. User queries are converted into dense vector representations using a pre-trained sentence embedding model, and the system retrieves the most relevant information from a pre-indexed document collection stored in an FAISS vector database. This approach ensures efficient and scalable similarity search across large volumes of support documents.

Instead of relying on traditional keyword matching, the system understands the meaning of user queries, allowing it to handle wording variations and retrieve contextually relevant information. The retrieved document chunks are then processed and presented as structured, human-readable responses, ensuring clarity and usability.

The system strictly follows a retrieval-driven approach, where all answers are grounded in existing data, minimizing incorrect or hallucinated responses. By separating document retrieval from answer presentation, it ensures reliability and consistency in customer support interactions. Overall, the assistant functions as an intelligent product knowledge engine that improves support efficiency, reduces response time, and enhances user experience by delivering accurate and context-aware answers derived from real support data.

2. Problem Statement:

Most existing search systems and AI-based assistants in e-commerce platforms rely heavily on keyword-based retrieval mechanisms, which often fail to understand the actual intent behind user queries. While these systems may return results based on exact keyword matches, they struggle to retrieve relevant information when queries are phrased differently, use synonyms, or when product data contains inconsistent terminology across documents. This limitation reduces the effectiveness of search results, especially in dynamic e-commerce environments where product descriptions and user queries vary widely.

Furthermore, traditional keyword-based approaches cannot capture semantic relationships between terms. As a result, even slightly complex or conversational queries can lead to irrelevant or incomplete results. For example, a user asking about “laptop battery life issues” may not receive accurate results if the system only matches exact keywords like “battery” or “power,” ignoring the broader context of the query.

Although some AI assistants can generate fluent and human-like responses, they often lack grounding in verified product data. This can result in inaccurate or misleading answers, especially when the system relies purely on generative capabilities without proper retrieval support. Such responses may include hallucinated information, incorrect product specifications, or outdated details, which can negatively impact user trust.

This issue becomes more critical in scenarios involving sensitive or decision-based queries, such as warranty conditions, compatibility specifications, refund timelines, or return policies. In such cases, even minor inaccuracies can lead to customer dissatisfaction, financial loss, or operational inefficiencies.

As a result, both customer support agents and end-users face challenges in trusting automated systems, leading to an increased dependency on manual document searching and human intervention. This not only reduces efficiency but also increases response time and operational costs for businesses.

Overall, the lack of semantic understanding, contextual retrieval, and reliable verification mechanisms significantly limits the effectiveness of traditional systems in handling real-world e-commerce queries. Therefore, there is a strong need for more advanced, AI-driven solutions that combine semantic search with grounded knowledge retrieval to provide accurate, context-aware, and trustworthy responses.

3. Objectives:

The primary objective of this project is to develop an AI-driven product knowledge system that delivers accurate, reliable, and context-aware responses by leveraging semantic search over a structured support knowledge base. Unlike traditional keyword-based systems, this approach focuses on understanding the intent behind user queries rather than relying solely on exact word matches. By utilizing semantic similarity, the system ensures that relevant information is retrieved even when queries are phrased differently or contain varied terminology.

Another key objective is to design a robust, retrieval-focused architecture in which user queries are transformed into vector embeddings and compared against a pre-indexed document collection. This collection is stored in a FAISS (Facebook AI Similarity Search) vector database, enabling efficient similarity-based search. To further enhance performance, the system employs the HNSW (Hierarchical Navigable Small World) indexing mechanism within FAISS, which allows for fast, scalable, and approximate nearest neighbour search across large datasets, making it suitable for real-world e-commerce applications.

The project also emphasizes ensuring that all generated responses are grounded in existing and verified data sources. By integrating retrieval with response generation, the system minimizes the risk of hallucinated or incorrect outputs. Retrieved document chunks are processed, ranked, and presented in a structured and user-friendly format, improving readability and usability for both customers and support agents.

In addition, the system aims to provide a seamless and intuitive user experience through natural language interaction. Users can ask product-related queries in simple, conversational language and receive precise, meaningful, and contextually relevant responses. The system is designed to handle key e-commerce use cases, including return policies, refund timelines, warranty claims, product compatibility, and order-related queries, thereby enhancing customer support efficiency. Finally, the system is built with a modular and scalable design, allowing easy integration of additional datasets, advanced retrieval techniques, or more sophisticated AI models in the future. This flexibility ensures that the system can evolve with changing requirements and growing data volumes. Overall, the project demonstrates how technologies such as Python, Flask, FAISS with HNSW indexing, and sentence embeddings can be effectively combined to build a practical, efficient, and intelligent product knowledge system for modern e-commerce platforms.

4. Core Technologies Used

Technology	Purpose
Python	Backend logic, data processing, and semantic retrieval pipeline implementation
Flask	API development for handling user queries and communication between the frontend and backend
FAISS	Vector database for efficient similarity search and document retrieval
HNSW Index (via FAISS)	enables fast approximate nearest neighbour search using graph-based indexing
Sentence Transformers	generates embeddings for product/support documents and user queries
NumPy	handles numerical operations for embeddings and similarity computations
Datasets (Hugging Face)	Used to load and manage the customer support QA dataset
VectorEmbedding Model	Converts text data into dense vectors for semantic search
Retrieval-Based Architecture	Retrieves relevant documents and presents structured answers without hallucination

4. Key Features

Semantic Search Engine

- Uses vector embeddings to understand the meaning of queries instead of relying on keyword matching
- Retrieves relevant support information even when query wording differs

Natural Language Query Processing

- Allows users to ask product and support-related questions in simple English
- Converts queries into embeddings for semantic similarity matching

Retrieval-Based Response System

- Retrieves relevant document chunks from a pre-indexed knowledge base
- Presents answers directly from retrieved data without generating unsupported content

Document Processing & Indexing

- Processes structured FAQ-based support data
- Splits content into smaller chunks for efficient semantic retrieval

Efficient Vector Search using FAISS with HNSW

- Uses FAISS for fast similarity search
- Implements HNSW indexing for scalable and efficient nearest neighbour retrieval

Context-Aware Answer Presentation

- Uses top-retrieved document chunks to form relevant responses
- Ensures answers are accurate and based on actual data

Fast and Scalable Retrieval

- Optimised for quick response even with large datasets
- Supports efficient search using approximate nearest neighbour techniques

Multi-Document Context Handling

- Retrieves multiple relevant results for a query
- Combines them into a structured and meaningful response

Output Cleaning & Formatting

- Removes unnecessary metadata and irrelevant content
- Displays clean, readable, and user-friendly responses

Modular and Scalable Design

- Easily extendable with new datasets and features
- Designed to scale for real-world customer support systems

5. Target Users

a. Customer support agents handling product-related queries in e-commerce platforms
b. E-commerce companies managing large-scale product documentation
c. Product managers responsible for maintaining and accessing product knowledge
d. Technical support teams assisting users with product compatibility and specifications
e. Online retail users seeking quick and accurate product information
f. Organisations looking to automate knowledge retrieval and improve customer experience

6. Real-World Applications

RetailMind AI bridges the gap between traditional keyword-based search systems and intelligent semantic retrieval by combining natural language understanding with vector-based document search and AI-driven response generation. It enables users and customer support teams to access accurate product information quickly by providing context-aware answers derived from existing documentation. The system is highly relevant in modern e-commerce environments, where users expect fast, reliable, and precise responses to their queries.

APPLICATION SCENARIOS / USE CASES:

Scenario 1: Customer Support Query Resolution

A customer support agent receives a query regarding product warranty or return conditions. Instead of manually searching through multiple documents, the system retrieves the most relevant information from the knowledge base and generates an accurate, easy-to-understand response. This significantly reduces response time and improves customer satisfaction.

Scenario 2: Product Compatibility Check

A user wants to know whether a specific product is compatible with another device or accessory. The system analyses product specifications from multiple documents, retrieves the relevant details, and provides a precise answer. This helps users make informed purchasing decisions without confusion.

Scenario 3: Product Information Retrieval

A customer asks detailed questions about product features, materials, or usage instructions. The system extracts relevant content from product manuals and descriptions, then generates a clear explanation, ensuring users receive complete and accurate information.

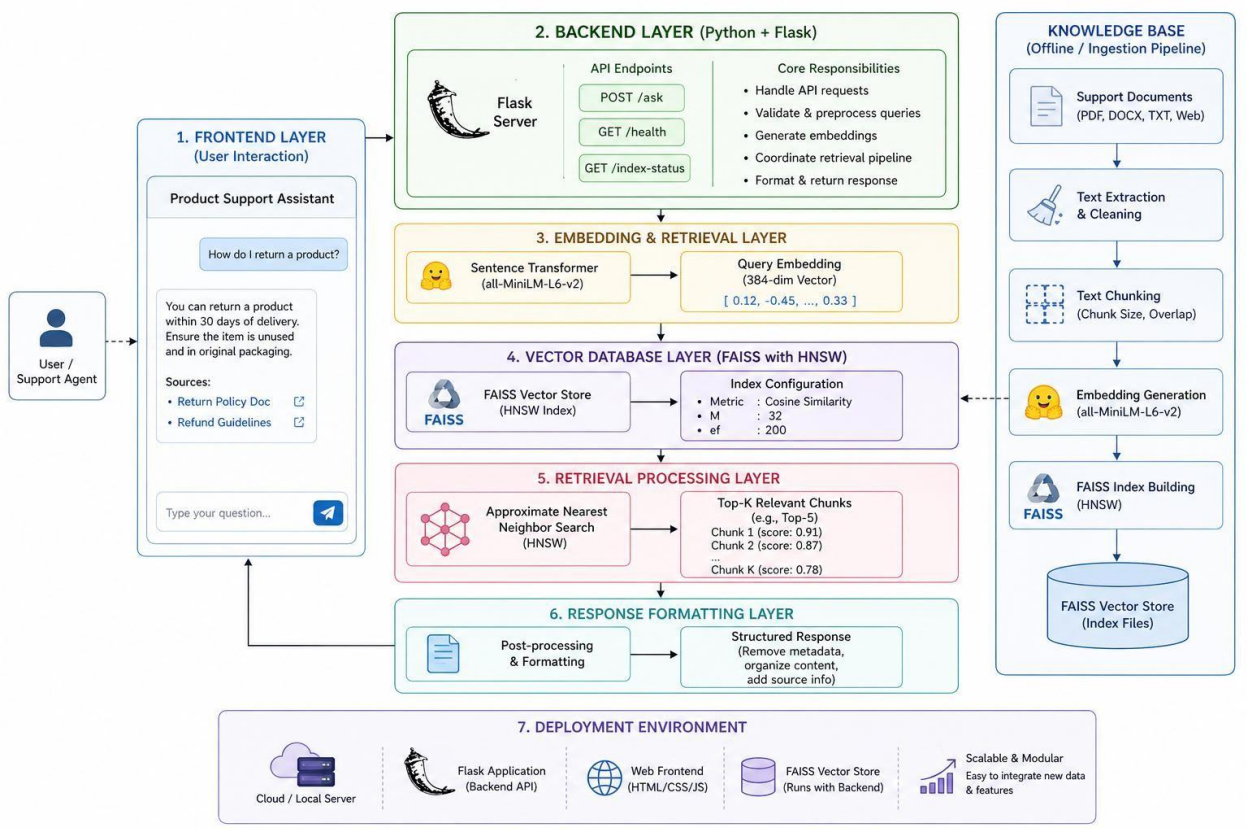
Scenario 4: Knowledge Management for E-Commerce Companies

E-commerce organisations can use the system to manage large volumes of product data efficiently. Instead of relying on manual document searches, employees can query the system to retrieve specific information instantly, improving operational efficiency and reducing workload.

Scenario 5: AI-Powered Chat Support Integration

RetailMind AI can be integrated into chatbots or customer service platforms to provide automated responses to product-related queries. This enhances customer experience by offering instant, accurate, and context-aware support without human intervention.

TECHNICAL ARCHITECTURE :



Frontend Layer

The frontend acts as the user interaction interface where users or customer support agents can ask product-related queries in natural language. It provides a simple chat-based UI to input questions related to warranty, return policies, compatibility, and order issues. The interface displays retrieved answers in a clean and user-friendly format, ensuring an intuitive experience.

Backend Layer

The backend is developed using Python and Flask, serving as the central controller of the system. It handles API requests, processes user queries, generates embeddings, and coordinates

communication between the vector database and response module. It ensures smooth execution of the semantic retrieval pipeline.

Embedding & Retrieval Layer

This layer converts user queries and support documents into dense vector embeddings using Sentence Transformers (all-MiniLM-L6-v2). These embeddings capture the semantic meaning of text, allowing the system to understand intent rather than relying on keyword matching. The query embedding is compared with stored embeddings to retrieve the most relevant document chunks.

Vector Database Layer (FAISS with HNSW)

The FAISS-based vector store is used to index and retrieve high-dimensional embeddings efficiently. The system uses HNSW (Hierarchical Navigable Small World) indexing within FAISS to perform fast approximate nearest neighbour search. This enables scalable and efficient retrieval across large support datasets.

Retrieval Processing Layer

This layer handles the retrieval of top-k relevant document chunks based on semantic similarity. Instead of generating new content, the system selects the most relevant existing information from the knowledge base. This ensures that all responses are grounded in real data and reduces the chances of incorrect outputs

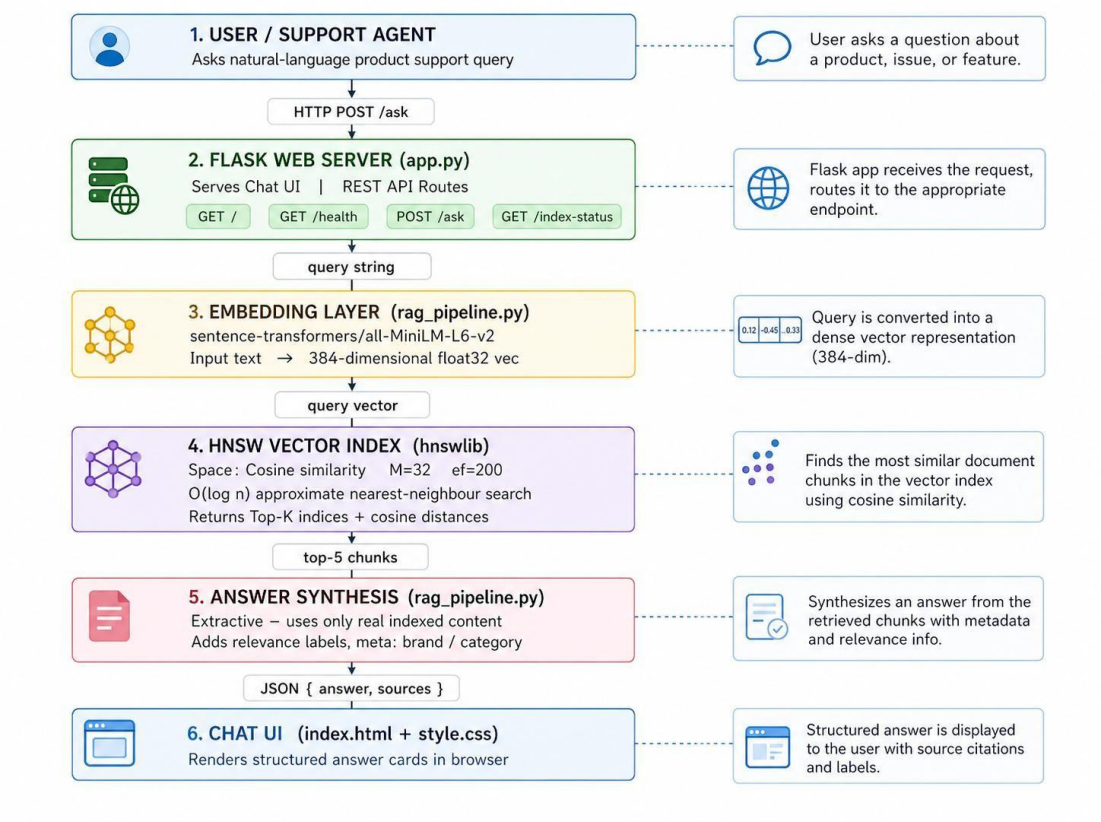
Response Formatting Layer

The retrieved results are processed and formatted into a structured, human-readable response. This layer removes unnecessary metadata, organises the content, and ensures clarity, making the output easy to understand for users.

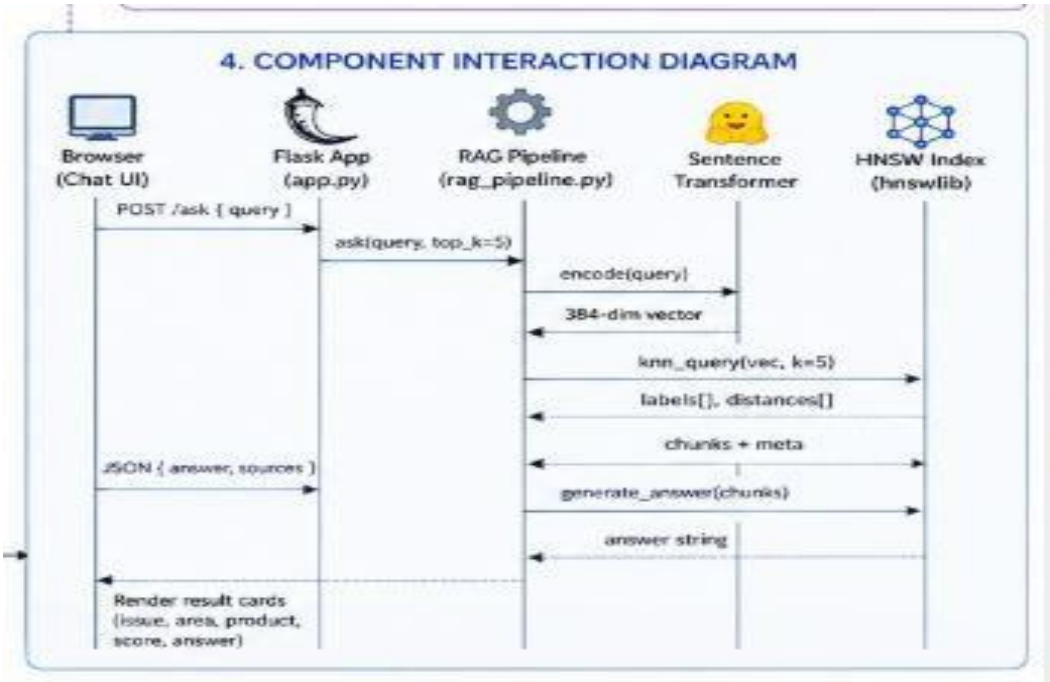
Deployment Environment

The application can be deployed locally or on cloud platforms. The backend runs on a Flask server, the frontend is served as a web interface, and the FAISS vector index operates alongside the backend for real-time retrieval. The system is designed to be modular and scalable, allowing easy integration of additional datasets and features.

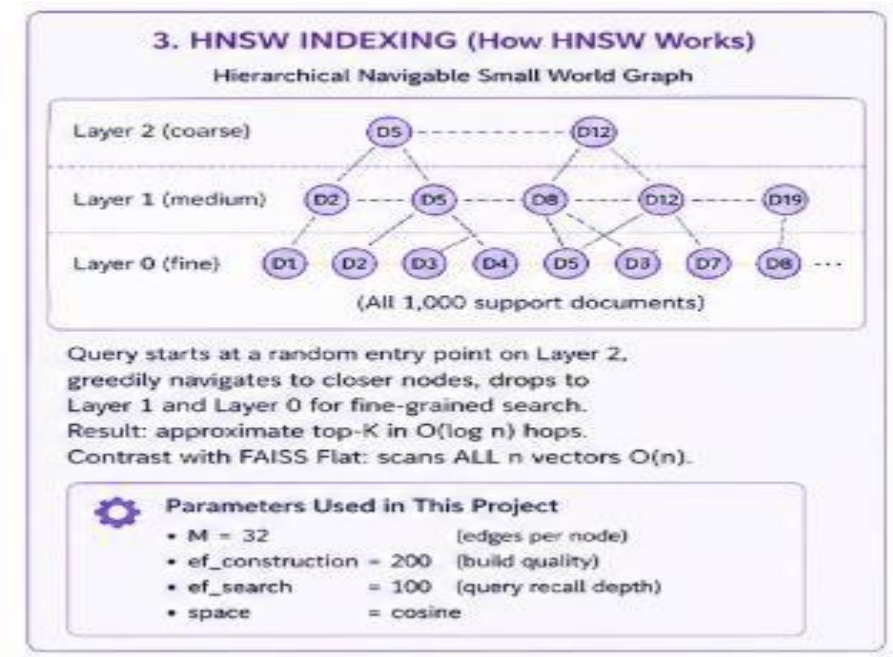
High-Level Architecture Diagram

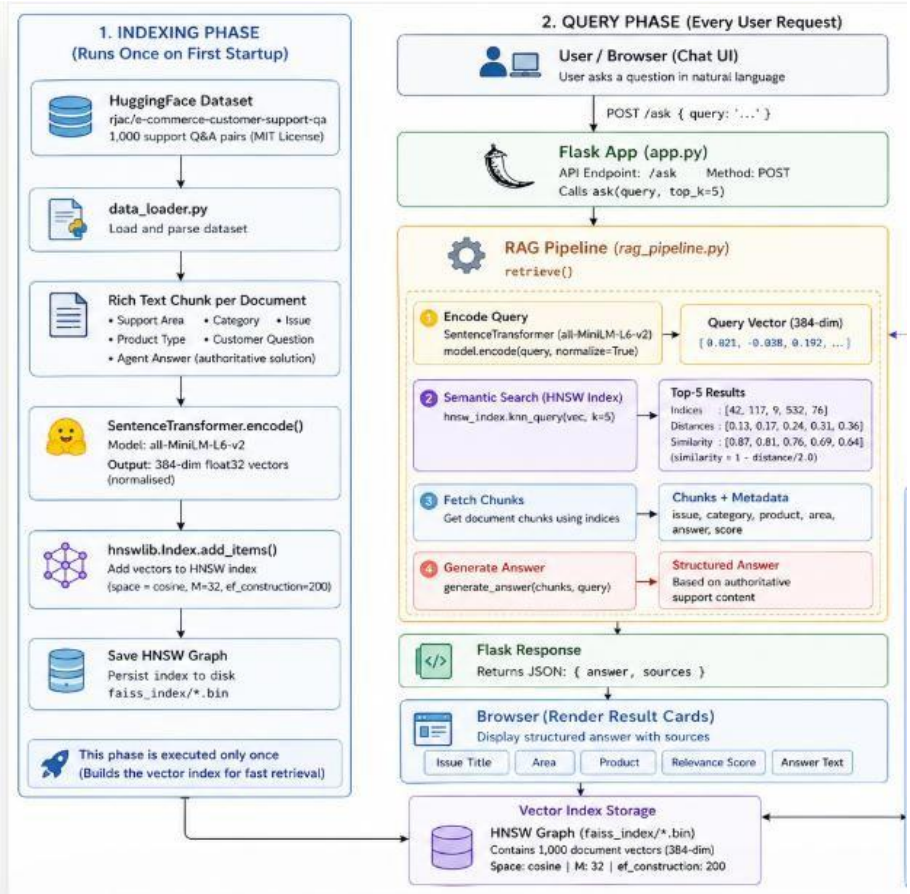


Component Interaction Diagram



HNSW INDEX AND RAG PIPELINE:





Pre-requisites:

Software Requirements

● **Python**: Version 3.8 or higher (recommended Python 3.10) for backend development, embedding generation, HNSW indexing, and RAG pipeline execution.

● **pip (Python Package Manager)**: Used to install and manage dependencies such as Flask, hnswlib, sentence-transformers, and datasets.

● **IDE**: Visual Studio Code (recommended) with extensions like Python, Pylance for efficient coding, debugging, and project management.

● **Browser**: Modern web browsers such as Chrome, Edge, or Firefox for testing the web interface and API responses.

● **Git**: A version control system for managing source code, collaboration, and version tracking.

Libraries / Frameworks / Dependencies

Backend Dependencies (Python)

Install using:
pip install flask hnswlib sentence-transformers datasets numpy

● **Flask**: Lightweight Python framework used to build REST APIs and handle user queries.
● **hnswlib**: Efficient vector similarity search library implementing HNSW (Hierarchical Navigable Small World) indexing for fast retrieval.
● **Sentence Transformers**: Used to convert user queries and documents into dense vector embeddings (384-dimensional).
● **Datasets (HuggingFace)**: Used to load the customer-support dataset for building the knowledge base.
● **NumPy**: Used for numerical computations and vector operations.

Frontend Dependencies (React)

- **HTML/CSS: Used to design a simple,** user-friendly chat interface.

● **JavaScript**: Handles API calls to the backend and dynamically displays results.
● **Flask Templates (Jinja2)**: Used to render dynamic web pages (index.html).

AI / RAG Integration

● **Embedding Model (all-MiniLM-L6-v2)**: Generates 384-dimensional vector representations for semantic understanding.
● **HNSW Vector Index (hnswlib)**: Stores embeddings and enables fast approximate

nearest neighbour search.

- **Retrieval-Augmented Generation (RAG)**: Combines semantic retrieval with extractive answer synthesis (no hallucination).
- **Extractive Answer Generation**: Generates responses strictly from retrieved documents (no external LLM required).

Hardware Requirements

- **CPU**: Minimum Intel Core i5 or equivalent processor for embedding computation and backend execution.
- **RAM**: Minimum 8 GB recommended for efficient indexing and retrieval.
- **Storage**: At least 2–3 GB free space for dataset, embeddings, and index files.
- **GPU (Optional)**: Not required; CPU-based inference is sufficient.
- **Network**: Required for downloading the dataset from HuggingFace during the first run.

PRIOR KNOWLEDGE REQUIRED

1. Programming Knowledge

- Strong understanding of Python fundamentals (functions, loops, lists, dictionaries)
- Knowledge of working with Python libraries and modules
- Basic text preprocessing and data handling techniques
- Debugging and exception handling

2. Framework & Development Basics

- Understanding of Flask framework for REST API development
- Knowledge of HTTP requests and responses
- Familiarity with HTML, CSS, and JavaScript for frontend
- API routing and request handling

3. API & Backend Concepts

- Understanding REST APIs and HTTP methods (GET, POST)
- Client-server communication flow
- JSON data handling and response formatting

4. AI / RAG & NLP Fundamentals

- Basic understanding of embeddings and semantic search
- Knowledge of vector similarity concepts
- Understanding of Retrieval-Augmented Generation (RAG)
- Awareness of LLM limitations (hallucination) and importance of grounded responses

5. Data & Domain Knowledge

- Understanding product-related data (FAQs, support queries)
- Handling structured and unstructured data
- Knowledge of customer support workflows

6. System Design Concepts

- Layered architecture (frontend, backend, vector database, AI layer)
- Separation of concerns (retrieval vs response generation)
- Basic knowledge of vector databases and similarity search
- Modular and scalable system design

PROJECT OBJECTIVES

Technical Objectives

- Design and develop a RAG-based product knowledge system using Python and Flask
- Implement semantic search using embeddings and HNSW indexing
- Build a document processing pipeline to handle customer support data
- Develop a retrieval pipeline using vector similarity search
- Implement extractive answer generation using retrieved content
- Ensure clear separation between retrieval and response generation

Performance Objectives

- Achieve high retrieval accuracy using semantic similarity
- Maintain low latency for real-time query responses
- Efficiently handle large datasets using HNSW indexing
- Provide consistent and relevant responses

Deployment Objectives

- Deploy backend using Flask API
- Store vector index locally for fast access
- Enable modular scalability for adding new datasets
- Ensure easy setup and execution

Learning Outcomes

- Gain practical experience in building RAG-based systems
- Understand embeddings, vector search, and semantic retrieval
- Learn how to design AI systems without hallucination
- Develop backend skills using Flask
- Improve knowledge of system architecture and API design

Project Workflow:

Milestone 1: Requirement Analysis & Design

- Problem definition
- Functional & non-functional requirements
- System architecture design
- Technology selection

Milestone 2: Environment Setup

- Install Python and dependencies
- Set up project structure
- Configure environment

Milestone 3: Data Processing & Indexing

- Load dataset using HuggingFace
- Clean and preprocess data
- Create rich text chunks
- Generate embeddings
- Build HNSW index

Milestone 4: Retrieval System

- Encode user queries
- Perform HNSW similarity search
- Retrieve top-k relevant chunks

Milestone 5: RAG Pipeline Development

- Integrate retrieval with answer generation
- Format structured responses
- Optimise relevance scoring

Milestone 6: System Integration

- Connect frontend with backend APIs
- Integrate full pipeline
- Improve response time

Milestone 7: Testing & Validation

- Test retrieval accuracy
- Validate end-to-end pipeline
- Perform integration testing
- Evaluate performance

Milestone 8: Deployment

- Deploy Flask backend
- Serve frontend UI
- Monitor performance
- Maintain and update the system

Milestone 1: Requirement Analysis & System Design

Activity 1.1: Problem Definition

Identify the need for an intelligent product support system that can understand user queries in natural language and provide accurate, context-aware answers from a large knowledge base. Traditional keyword-based search systems fail to retrieve relevant information due to variations in wording and lack of semantic understanding. This results in inefficient customer support and delays in finding correct information.

Design a solution using **semantic search and Retrieval-Augmented Generation (RAG)** that understands the meaning of user queries instead of relying on exact keyword matches.

The system should convert both user queries and product support documents into vector embeddings and use **HNSW-based similarity search** to retrieve the most relevant document chunks.

Ensure that the system generates **accurate and grounded responses** by using only retrieved knowledge base content, thereby eliminating hallucination.

The solution should be scalable, efficient, and capable of handling real-world queries such as:

- Return policies
- Warranty details
- Order tracking issues

- Product compatibility

The goal is to build a **semantic product knowledge engine** that improves support efficiency and delivers precise and reliable answers.

Activity 1.2: Functional Requirements

FR-01: Allow users to input product-related queries in natural language through a chat-based interface.
FR-02: Validate and process user queries before passing them to the system.
FR-03: Convert user queries into vector embeddings using Sentence Transformers.
FR-04: Retrieve top-k relevant document chunks using HNSW similarity search.
FR-05: Perform semantic search instead of keyword-based matching.
FR-06: Generate structured and human-readable responses from retrieved data.
FR-07: Display answers along with sources and relevance scores.
FR-08: Provide REST API endpoints for query handling and system status.
FR-09: Handle empty or invalid queries with proper error messages.
FR-10: Ensure all responses are generated only from existing knowledge base (no hallucination).

Activity 1.3: Non-Functional Requirements

NFR-01 Performance:
System should provide fast response times using efficient HNSW indexing.
NFR-02 Accuracy:
Ensure high retrieval accuracy using semantic similarity search.
NFR-03 Reliability:
The system should consistently return correct, structured responses.
NFR-04 Scalability:
Architecture should support large datasets and increased query load.
NFR-05 Security:
System should safely handle user inputs and prevent invalid queries.
NFR-06 Maintainability:
Follow modular design separating frontend, backend, and RAG pipeline.
NFR-07 Usability:
Provide simple and intuitive user interface for interaction.
NFR-08 Robustness:
Handle missing or incorrect inputs gracefully.

Activity 1.4: System Design Decisions

Semantic RAG Architecture

The system follows a Retrieval-Augmented Generation approach where relevant documents are retrieved first and then used to generate responses. This ensures accuracy and avoids hallucinated outputs.

HNSW Indexing
The system uses HNSW (Hierarchical Navigable Small World) for fast approximate nearest neighbour search with O(log n) complexity, making retrieval efficient and scalable.

Embedding-Based Search
All queries and documents are converted into dense vector embeddings (384-dimensional) using Sentence Transformers to capture semantic meaning.

Extractive Answer Generation
The system generates responses only from retrieved documents instead of creating new content, ensuring factual correctness.

Modular Architecture
The system is divided into:
- Frontend (UI)
- Backend (Flask API)
- Data Processing Layer
- RAG Pipeline
- Vector Index

API-Driven Design
All functionalities are exposed through REST APIs such as:
- /ask for query processing
- /health for system status
- /index-status for index information

Background Indexing
The vector index is built once during startup and reused for faster query processing.

Input Validation & Error Handling
The system validates user inputs and handles errors gracefully to ensure smooth operation.

Activity 1.5: Technology Stack Selection

Frontend Technologies
- HTML, CSS for UI design
- JavaScript for API interaction
- Flask Templates (Jinja2) for dynamic rendering

Backend Technologies
- Python for backend logic
- Flask for REST API development

AI & NLP Technologies

- Sentence Transformers (all-MiniLM-L6-v2) for embeddings
- Semantic search for query understanding

Vector Database

- hnswlib for HNSW-based similarity search

Dataset

- HuggingFace dataset: rjac/e-commerce-customer-support-qa

Storage

- Local storage for index files and processed data

Milestone 2: Environment Setup & Initial Configuration

Activity 2.1: Development Environment Setup

● Install Python (version 3.8 or higher, recommended 3.10)
● Verify installation:
python --
version pip --
version
● Install Visual Studio Code with the following extensions:
- Python
- Pylance

● Ensure a modern web browser (Chrome, Edge, or Firefox) is available for testing
● Install Git for version control and project management

Activity 2.2: Dependency Installation

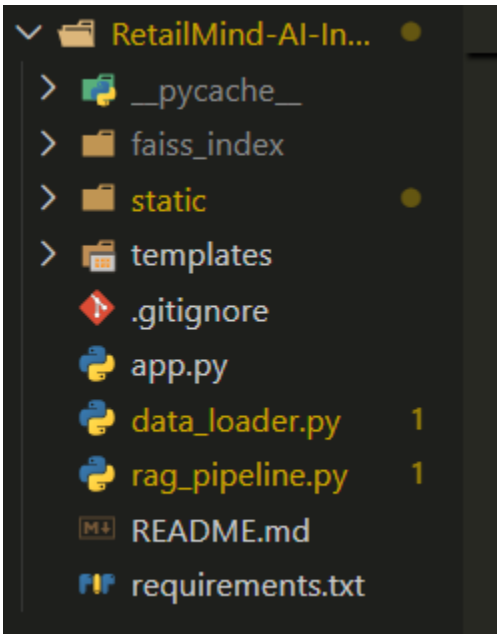
Install all required project dependencies using **pip** to support backend development, semantic search, and RAG pipeline execution.
The backend dependencies include **Flask** for building REST APIs, **hnswlib** for efficient vector similarity search using HNSW indexing, and **Sentence Transformers** for generating embeddings from user queries and documents.
The system also uses the **HuggingFace datasets library** to load the customer-support dataset and **NumPy** for handling numerical operations related to embeddings.
Install all dependencies using the following command:
pip install flask hnswlib sentence-transformers datasets numpy
These libraries enable:

- API handling and request processing
- Embedding generation for semantic understanding
- Efficient vector storage and retrieval
- Dataset loading and preprocessing

No external LLM APIs or Node.js-based dependencies are required, as the system uses **extractive RAG** instead of generative models.

Activity 2.3: Project Structure Creation

Organize the project into a clean and modular folder structure by separating backend logic, data processing, and user interface components. This improves code readability, maintainability, and scalability.
The project structure is designed to clearly divide responsibilities such as API handling, data loading, retrieval logic, and frontend rendering.
Project Structure



Activity 2.4: Configuration Setup

The configuration includes specifying the embedding model (**Sentence Transformers – all-MiniLM-L6-v2**), vector dimensions, and HNSW indexing parameters such as graph connectivity (M), construction efficiency (ef_construction), and search efficiency (ef_search). These settings are defined within the backend to control embedding generation and similarity search performance.

Backend configurations also include defining API routes, request handling logic, query validation rules, and response formatting mechanisms using the Flask framework. The system ensures that all incoming queries are validated and processed before being passed to the retrieval pipeline.

The RAG pipeline is initialized during application startup, where the system checks for an existing vector index. If not found, it automatically loads the dataset, generates embeddings, and builds the HNSW index. This initialization ensures that the system is ready for real-time query processing.

All components—frontend interface, Flask backend, data processing module, and vector index—are properly configured and connected to enable seamless communication and efficient execution of user queries.

Milestone 3: Backend Development (RAG System)

Activity 3.1: Setup Flask Server and API Routes

The backend server is developed using **Python and Flask** to handle API requests and manage system operations. The Flask application is initialized to listen for incoming HTTP requests and serve as the central controller of the system.

API routes are defined to handle different functionalities such as rendering the user interface, processing user queries, checking system health, and monitoring index status. The primary route (/ask) accepts user queries in JSON format and forwards them to the RAG pipeline for processing.

The backend processes each query by invoking the retrieval and answer generation functions, ensuring that the response is based on semantically relevant document chunks. The results are then formatted into a structured JSON response and sent back to the frontend.

Additional routes such as /health and /index-status are implemented to monitor system performance and readiness. These endpoints help ensure reliability and provide insights into the backend state.

This Flask-based setup establishes a structured communication flow between the frontend interface and the RAG pipeline, forming the foundation for efficient query handling and response generation.

```python
@app.route("/")
def index():
    return render_template("index.html")
@app.route("/health")
def health():
    return jsonify({"status": "ok", "index_ready": _index_ready})
@app.route("/index-status")
def index_status():
    if _index_error:
        return jsonify({"ready": False, "error": _index_error}), 500
    elapsed = round(time.time() - _load_start, 1)
    return jsonify({
        "ready":   _index_ready,
        "stats":   _index_stats,
        "elapsed": elapsed,
    })

@app.route("/ask", methods=["POST"])
def ask_route():
    # Guard: index not ready
    if not _index_ready:
        if _index_error:
            return jsonify({"error": f"Index failed: {_index_error}"}), 500
        elapsed = round(time.time() - _load_start, 1)
        return jsonify({
            "error": (
                f"Knowledge base is still loading ({elapsed}s elapsed). "
                "Please wait a moment and try again."
            )
        }), 503
```

Activity 3.2: Implement Embedding & Retrieval Service for Semantic Processing

This module is responsible for implementing the semantic understanding layer of the system using embedding-based retrieval instead of traditional LLM interaction.

The system uses **Sentence Transformers (all-MiniLM-L6-v2)** to convert both user queries and support documents into dense vector embeddings. These embeddings capture semantic meaning, allowing the system to understand user intent rather than relying on keyword matching.

The query embedding is compared with stored embeddings using **HNSW similarity search**, which retrieves the most relevant document chunks efficiently.

This module acts as a bridge between the backend API and the vector database by:

- Encoding user queries into vectors
- Performing similarity search
- Returning top-k relevant results

This approach ensures accurate, fast, and scalable retrieval while eliminating hallucination by grounding responses in real data.

```python
def _get_model():
    global _model
    if _model is None:
        print(f"[rag] Loading embedding model: {EMBED_MODEL}")
        _model = SentenceTransformer(EMBED_MODEL)
        dim = _model.get_sentence_embedding_dimension()
        print(f"[rag] Model ready — embedding dim: {dim}")
    return _model
```

```python
embeddings = model.encode(
    texts,
    batch_size=64,
    show_progress_bar=True,
    convert_to_numpy=True,
    normalize_embeddings=True,  # required for cosine space in hnswlib
).astype("float32")

# Initialise HNSW index
index = hnswlib.Index(space=HNSW_SPACE, dim=EMBED_DIM)
index.init_index(
    max_elements=n,
    ef_construction=HNSW_EF_CONSTRUCTION,
    M=HNSW_M,
)
```

Activity 3.3: Implement HNSW-Based Retrieval System

The retrieval system is implemented using **HNSW (Hierarchical Navigable Small World)** indexing, which enables fast approximate nearest neighbour search in high-dimensional vector space.

HNSW organizes embeddings into a multi-layer graph structure where:

- Upper layers provide coarse navigation
- Lower layers provide fine-grained search

This structure allows efficient traversal to find nearest neighbours in **O(log n)** time complexity.

The system retrieves the top-k most similar document chunks based on cosine similarity between query and stored embeddings.

HNSW parameters such as:

- **M (graph connectivity)**
- **ef_construction (index quality)**
- **ef_search (query accuracy)**

are configured to balance performance and accuracy.

```python
def retrieve(query, top_k=5):
    """
    Encode the query, run HNSW approximate nearest-neighbour search,
    and return the top_k most semantically similar support documents.

    HNSW returns distances in cosine space (0 = identical, 2 = opposite).
    We convert to similarity: sim = 1 - (distance / 2)  → [0, 1].

    Parameters
    ----------
    query  : str   natural-language customer question
    top_k  : int   number of results (default 5)

    Returns
    -------
    list[dict] — sorted by descending similarity
    """
    ensure_index()
    model = _get_model()

    q_vec = model.encode(
        [query],
        convert_to_numpy=True,
        normalize_embeddings=True,
    ).astype("float32")
```

```
    # knn_query returns (labels, distances)
    labels, distances = _index.knn_query(q_vec, k=min(top_k, _index.get_current_count()))

    results = []
    for label, dist in zip(labels[0], distances[0]):
        # Convert cosine distance → similarity score [0,1]
        similarity = float(1.0 - dist / 2.0)
        chunk = _chunks[label]
        results.append({
            "text":   chunk["text"],
            "source": chunk["source"],
            "score":  round(similarity, 4),
            "meta":   chunk.get("meta", {}),
        })

    return results
```

Activity 3.4: Implement Answer Generation Module

The answer generation module formats the retrieved document chunks into a structured and human-readable response.

Instead of generating new content, the system uses an **extractive approach**, where answers are directly taken from the retrieved knowledge base. This ensures factual correctness and avoids hallucinated outputs.

The response includes:

- Relevant issue/topic
- Product information
- Extracted answer
- Relevance score

This structured format improves readability and helps users understand the context of the answer.

```
def generate_answer(query, context_chunks):
    """
    Synthesises a structured answer from the retrieved support documents.
    Every word comes directly from the indexed dataset — no hallucination.

    Format:
      Header line → per-result block (area, category, product, answer)
    """
    if not context_chunks:
        return (
            "No relevant support documents were found for your query.\n\n"
            "Try rephrasing — for example, mention the product category, "
            "the specific issue (warranty, return, compatibility), or "
            "describe the problem in more detail."
        )

    n = len(context_chunks)
    lines = [
        f"**Found {n} relevant support document{'s' if n > 1 else ''} for:** \"{query}\"",
        "",
        "---",
        "",
    ]
```

```
for i, chunk in enumerate(context_chunks, 1):
    meta       = chunk.get("meta", {})
    issue_area = meta.get("issue_area", "—")
    category   = meta.get("category",   "—")
    issue      = meta.get("issue",      "—")
    product    = meta.get("product",    "—")
    complexity = meta.get("complexity", "—")
    answer     = meta.get("answer",     "")
    score      = chunk["score"]
    rel_label  = _relevance_label(score)

    # Truncate very long answers for readability
    if len(answer) > 450:
        answer = answer[:447] + "..."

    lines.append(f"**{i}. {issue}**")
    lines.append(f"Area: {issue_area}  ·  Category: {category}  ·  Product: {product}")
    lines.append(f"Complexity: {complexity}  ·  Relevance: {rel_label} ({score:.2f})")
    lines.append(f"Answer: {answer}")
    lines.append("")

return "\n".join(lines)
```

Activity 3.5: Integrate Full RAG Pipeline

The complete RAG pipeline integrates embedding generation, retrieval, and answer formatting into a single workflow.

The process flow:

1. User query is received
2. Query is converted into an embedding
3. HNSW retrieves the top-k relevant documents
4. Answer is generated from retrieved content
5. Response is returned to the user

This pipeline ensures efficient and accurate query handling in real-time.

```
def ask(query, top_k=5):
    """
    Full RAG pipeline: query → embed → HNSW retrieve → synthesise → return

    Returns dict with keys: answer, sources, chunks
    """
    chunks  = retrieve(query, top_k=top_k)
    answer  = generate_answer(query, chunks)
    sources = [c["source"] for c in chunks]
    return {
        "answer":  answer,
        "sources": sources,
        "chunks":  chunks,
    }
```

Milestone 4: Data Processing & Indexing

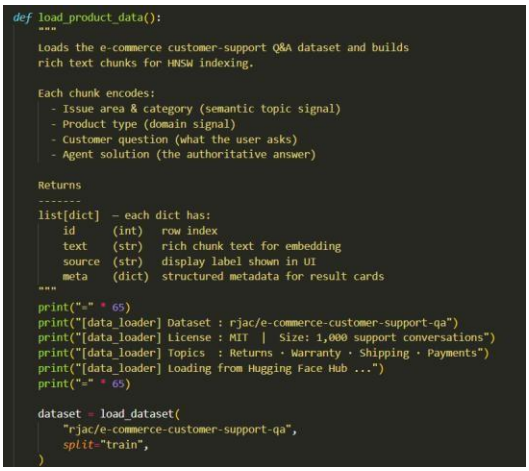
Activity 4.1: Data Loading and Preprocessing

The system loads the customer-support dataset using the HuggingFace datasets library. The data includes product-related queries and their corresponding solutions.

The preprocessing step involves:

- Extracting question-answer pairs
- Cleaning text data
- Removing duplicates
- Structuring data into meaningful chunks

Each chunk combines both query and solution to improve retrieval accuracy.



```
def load_product_data():
    """
    Loads the e-commerce customer-support Q&A dataset and builds
    rich text chunks for HNSW indexing.

    Each chunk encodes:
      - Issue area & category (semantic topic signal)
      - Product type (domain signal)
      - Customer question (what the user asks)
      - Agent solution (the authoritative answer)

    Returns
    -------
    list[dict]  — each dict has:
        id      (int)   row index
        text    (str)   rich chunk text for embedding
        source  (str)   display label shown in UI
        meta    (dict)  structured metadata for result cards
    """
    print("=" * 65)
    print("[data_loader] Dataset : rjac/e-commerce-customer-support-qa")
    print("[data_loader] License : MIT  |  Size: 1,000 support conversations")
    print("[data_loader] Topics  : Returns · Warranty · Shipping · Payments")
    print("[data_loader] Loading from Hugging Face Hub ...")
    print("=" * 65)

    dataset = load_dataset(
        "rjac/e-commerce-customer-support-qa",
        split="train",
    )
```

Activity 4.2: Document Chunking

The dataset is divided into smaller, meaningful chunks that combine:

- Issue category
- Product details
- Customer query
- Support answer

Chunking ensures that embeddings capture complete context and improves semantic retrieval performance.

```
chunks.append({
    "id":     i,
    "text":   full_text,
    "source": source,
    "meta": {
        "issue_area":  issue_area  or "General",
        "category":    issue_cat   or "-",
        "issue":       issue_sub   or issue_cat or "-",
        "product":     prod_sub    or prod_cat or "-",
        "complexity":  complexity  or "-",
        "question":    question,
        "answer":      solution,
    },
})
```

```
parts = []

if issue_area:
    parts.append(f"Support Area: {issue_area}")
if issue_cat:
    parts.append(f"Category: {issue_cat}")
if issue_sub:
    parts.append(f"Issue: {issue_sub}")
if prod_cat and prod_sub:
    parts.append(f"Product: {prod_sub} ({prod_cat})")
elif prod_cat:
    parts.append(f"Product Category: {prod_cat}")

parts.append(f"Customer Question: {question}")
parts.append(f"Support Answer: {solution}")

full_text = "\n".join(parts)
```

Activity 4.3: Embedding Generation

Each document chunk is converted into a vector embedding using Sentence Transformers.

These embeddings represent the semantic meaning of the text and are stored in the vector index for retrieval.

```
embeddings = model.encode(
    texts,
    batch_size=64,
    show_progress_bar=True,
    convert_to_numpy=True,
    normalize_embeddings=True,  # required for cosine space in hnswlib
).astype("float32")

# Initialise HNSW index
index = hnswlib.Index(space=HNSW_SPACE, dim=EMBED_DIM)
index.init_index(
    max_elements=n,
    ef_construction=HNSW_EF_CONSTRUCTION,
    M=HNSW_M,
)
```

Activity 4.4: Index Creation and Storage

The generated embeddings are stored in an HNSW index using hnswlib. The

index is saved locally to:

- Avoid recomputation
- Enable fast loading in future runs

Milestone 5: System Integration & Optimisation

Activity 5.1: Frontend and Backend Integration

The frontend interface is connected to the Flask backend using REST APIs.

User queries are sent via HTTP requests and responses are displayed dynamically on the UI.

```python
@app.route("/ask", methods=["POST"])
def ask_route():
    # Guard: index not ready
    if not _index_ready:
        if _index_error:
            return jsonify({"error": f"Index failed: {_index_error}"}), 500
        elapsed = round(time.time() - _load_start, 1)
        return jsonify({
            "error": (
                f"Knowledge base is still loading ({elapsed}s elapsed). "
                "Please wait a moment and try again."
            )
        }), 503
```

Activity 5.2: End-to-End Pipeline Integration

The system integrates all components:

- Frontend → Backend → RAG Pipeline → Response

This ensures smooth communication and execution of queries.

Activity 5.3: Performance Optimization

The system is optimized by:

- Using HNSW indexing for fast retrieval
- Loading index once at startup
- Limiting top-k results

These optimizations reduce latency and improve efficiency.

```python
def ensure_index():
    """
    Idempotent startup routine:
      - If index files exist on disk → load instantly
      - Otherwise → download dataset, embed, build & save HNSW index
    Called once in a background thread when Flask starts.
    """
    global _index, _chunks
    if _index is not None:
        return

    if os.path.exists(INDEX_PATH) and os.path.exists(CHUNKS_PATH):
        # Load chunks first to get count for hnswlib max_elements
        with open(CHUNKS_PATH, "rb") as f:
            _chunks = pickle.load(f)
        index = hnswlib.Index(space=HNSW_SPACE, dim=EMBED_DIM)
        index.load_index(INDEX_PATH, max_elements=len(_chunks))
        index.set_ef(HNSW_EF_SEARCH)
        _index = index
        print(f"[rag] HNSW index loaded — {_index.get_current_count():,} vectors")
    else:
        print("[rag] No index found — building from scratch...")
        _chunks = load_product_data()
        _index  = build_index(_chunks)
```

Activity 5.4: Error Handling and Validation

The system handles:

- Empty queries
- Long queries
- Invalid inputs

Proper error messages are returned to ensure usability.

Milestone 6: Testing & Validation

Activity 6.1: Functional Testing

Test the system with various queries related to:

- Returns
- Warranty
- Shipping
- Payments

Activity 6.2: Retrieval Accuracy Testing

Evaluate whether the system retrieves the most relevant documents based on query meaning.

Activity 6.3: Integration Testing

Verify end-to-end flow:
Query → Retrieval → Response

Activity 6.4: Performance

Evaluation

Measure:

- Response time
- Retrieval accuracy
- System reliability **Milestone**

7: Deployment

Activity 7.1: Backend

Deployment

Deploy the Flask application locally or on a server.

Activity 7.2: Frontend Deployment

Serve the UI through Flask templates.

Activity 7.3: Environment

Configuration

Ensure all dependencies and index files are properly configured.

Activity 7.4: Monitoring & Maintenance

Monitor system performance and update dataset or index when needed.

```
(rag_env) PS C:\Users\rishi\OneDrive\Desktop\AIML\RAG\RetailMind AI_project\RetailMind-AI-Intelligent-Product-Knowledge-Engine> p

[app] — Building HNSW index in background —

============================================================
  RetailMind AI  –  Semantic Product Knowledge Engine
  Problem: Warranty · Returns · Compatibility · FAQs
  Dataset: rjac/e-commerce-customer-support-qa (1k docs)
  Index  : HNSW via hnswlib (M=32, cosine similarity)
  Model  : sentence-transformers/all-MiniLM-L6-v2 (384-dim)
  URL    : http://127.0.0.1:5000
============================================================

[rag] HNSW index loaded – 932 vectors
[app] Index ready ✓  (0 documents indexed)
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
127.0.0.1 - - [30/Apr/2026 00:47:44] "GET /index-status HTTP/1.1" 200 -
Loading weights: 100%|| 103/103 [00:00<00:00, 1355.16it/s]
127.0.0.1 - - [30/Apr/2026 00:47:46] "GET /index-status HTTP/1.1" 200 -
127.0.0.1 - - [30/Apr/2026 00:47:48] "GET /index-status HTTP/1.1" 200 -
C:\Users\rishi\OneDrive\Desktop\AIML\RAG\RetailMind AI_project\RetailMind-AI-Intelligent-Product-Knowledge-Engine\rag_pipeline.py:71: FutureWarning: The `get_sentence_embeddi
g_dimension` method has been renamed to `get_embedding_dimension`.
  dim = _model.get_sentence_embedding_dimension()
[rag] Model ready – embedding dim: 384
```

PROJECT STRUCTURE

Clean Folder Structure

RetailMind-AI/

```
│
├── app.py              # Flask backend (API routes & server)
├── rag_pipeline.py     # Embedding, HNSW indexing, retrieval, answer generation
├── data_loader.py      # Dataset loading and preprocessing
├── requirements.txt    # Project dependencies
│
├── templates/
│   └── index.html      # Chat-based user interface
│
├── static/
│   └── style.css       # UI styling
│
└── faiss_index/
    ├── hnsw_support.bin     # Stored HNSW index
    └── support_chunks.pkl   # Stored document chunks
```

Folder / File Explanations

app.py

- Contains the backend logic built using Flask
- Handles API requests and routes
- Connects frontend with RAG pipeline
- Manages query validation and response handling
- Acts as the central controller of the application

rag_pipeline.py

- Implements the core RAG pipeline
- Handles embedding generation using Sentence Transformers
- Builds and loads HNSW vector index
- Performs semantic retrieval using similarity search
- Generates structured answers from retrieved documents
- Acts as the AI processing layer of the system

data_loader.py

- Loads dataset from HuggingFace
- Cleans and preprocesses product support data
- Extracts question-answer pairs

- Creates structured text chunks for embedding
- Prepares data for indexing and retrieval

templates/

- Contains frontend HTML files
- Provides chat-based user interface
- Displays user queries and system responses
- Uses Flask templating (Jinja2)

static/

- Contains CSS files for styling
- Improves UI appearance and layout
- Ensures responsive and clean design

faiss_index/

- Stores the vector database locally
- Contains HNSW index file (.bin)
- Stores processed chunks (.pkl)
- Enables fast loading and retrieval

requirements.txt

- Lists all Python dependencies
- Ensures easy installation and setup
- Maintains consistency across environments

RESULTS

System Output

The system successfully processes user queries related to product support and retrieves accurate, context-aware answers using semantic search.

It provides outputs such as:

- Return policy details
- Warranty information
- Shipping and order issues
- Product compatibility

The system uses embedding-based retrieval and HNSW indexing to ensure that responses are generated from the most relevant document chunks.

Unlike traditional AI systems, this approach ensures that all answers are **grounded in actual data**, eliminating hallucination and improving reliability.

The output is structured and human-readable, including:

- Issue/topic
- Product details
- Extracted answer
- Relevance score

Performance Evaluation

The application demonstrates efficient performance with fast response times due to optimized semantic retrieval using HNSW indexing.

Key performance aspects:

- Query processing is completed in milliseconds
- Retrieval is efficient with approximate nearest neighbor search (O(log n))
- Index is loaded once at startup, reducing runtime overhead
- System handles multiple queries consistently

The embedding model is lightweight and runs efficiently on CPU, ensuring smooth performance without requiring GPU support.

Screenshots

The system includes various interface and processing screenshots such as:

● Chat-based user interface showing query input

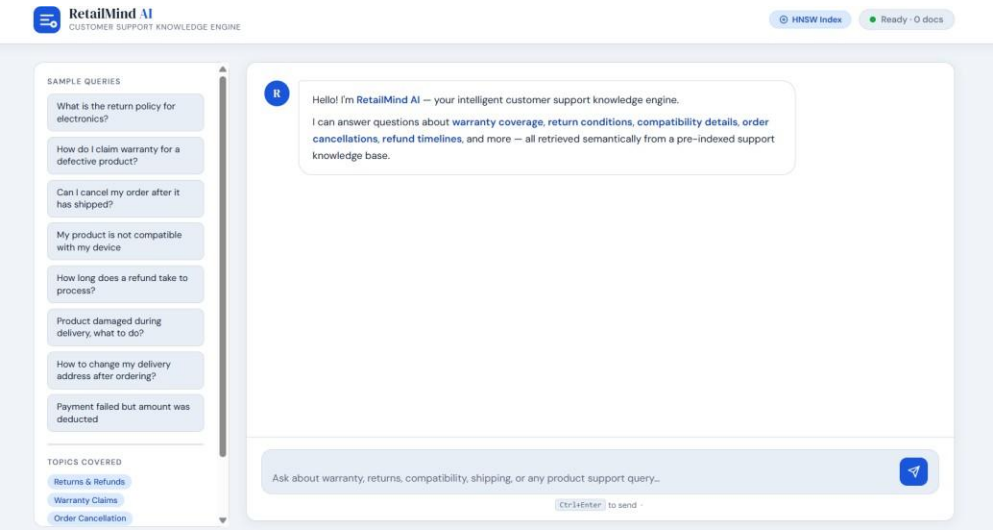
● API request handling and backend processing

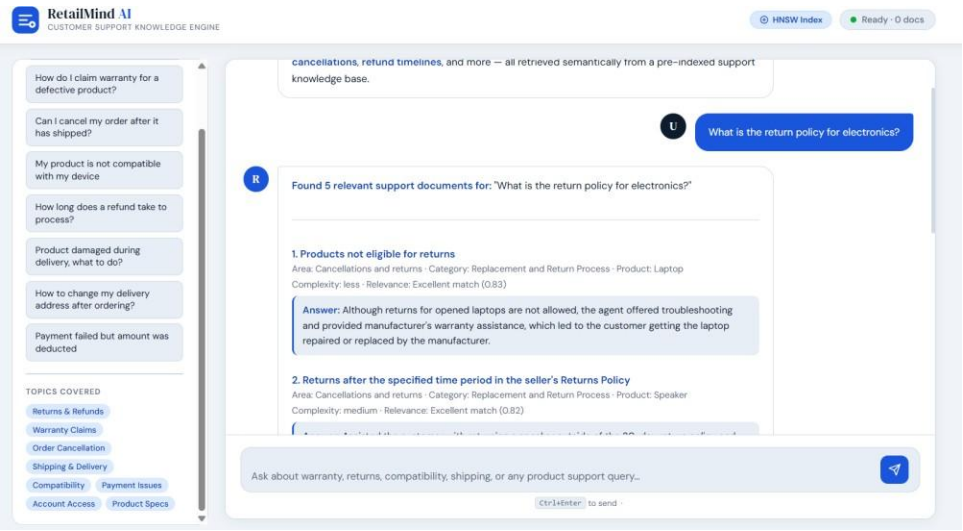
● Retrieval results with relevance scores

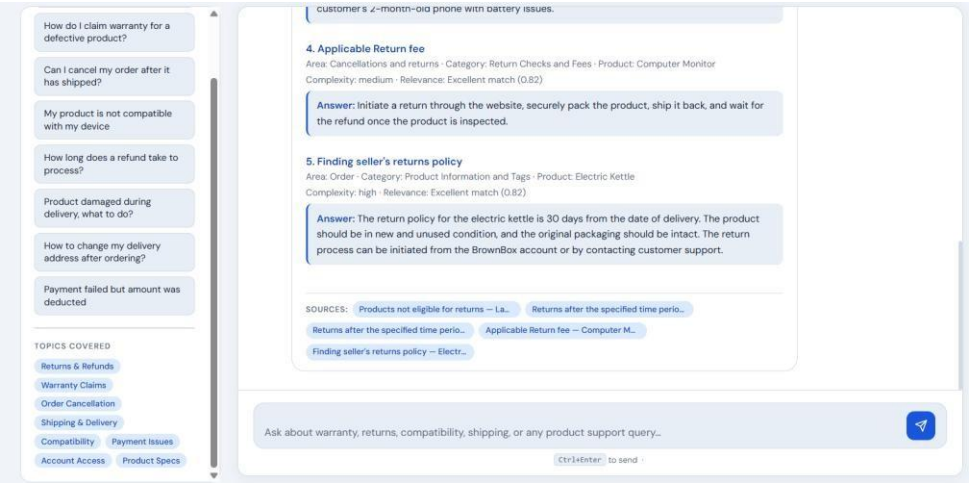
● Structured output displaying product support answers

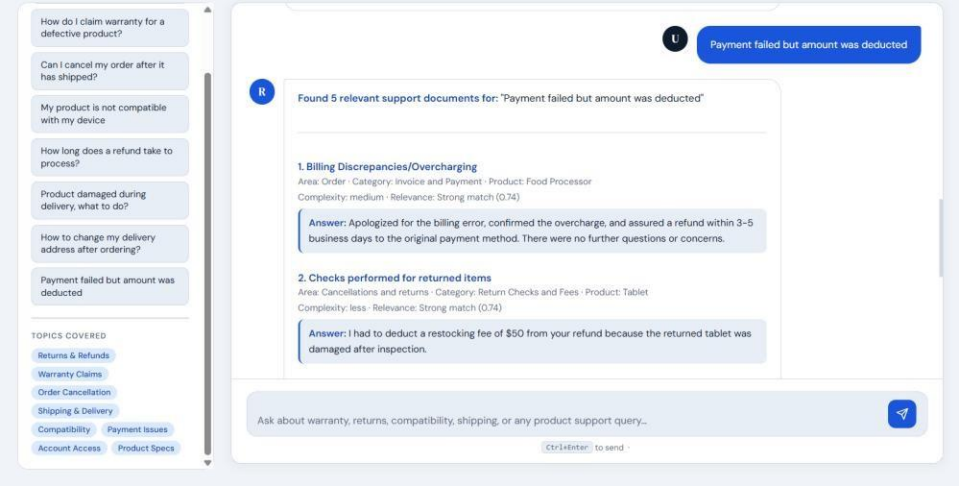
● Multiple query examples demonstrating system accuracy

These screenshots illustrate the complete workflow:


RetailMind AI
CUSTOMER SUPPORT KNOWLEDGE ENGINE
HNSW Index
Ready · 0 docs
SAMPLE QUERIES
What is the return policy for electronics?
How do I claim warranty for a defective product?
Can I cancel my order after it has shipped?
My product is not compatible with my device
How long does a refund take to process?
Product damaged during delivery, what to do?
How to change my delivery address after ordering?
Payment failed but amount was deducted
TOPICS COVERED
Returns & Refunds
Warranty Claims
Order Cancellation
Hello! I'm RetailMind AI — your intelligent customer support knowledge engine.
I can answer questions about warranty coverage, return conditions, compatibility details, order cancellations, refund timelines, and more — all retrieved semantically from a pre-indexed support knowledge base.
Ask about warranty, returns, compatibility, shipping, or any product support query...
Ctrl+Enter to send


RetailMind AI
CUSTOMER SUPPORT KNOWLEDGE ENGINE
HNSW Index
Ready · 0 docs
How do I claim warranty for a defective product?
Can I cancel my order after it has shipped?
My product is not compatible with my device
How long does a refund take to process?
Product damaged during delivery, what to do?
How to change my delivery address after ordering?
Payment failed but amount was deducted
TOPICS COVERED
Returns & Refunds
Warranty Claims
Order Cancellation
Shipping & Delivery
Compatibility
Payment Issues
Account Access
Product Specs
cancellations, refund timelines, and more — all retrieved semantically from a pre-indexed support knowledge base.
What is the return policy for electronics?
Found 5 relevant support documents for: "What is the return policy for electronics?"
1. Products not eligible for returns
Area: Cancellations and returns · Category: Replacement and Return Process · Product: Laptop
Complexity: less · Relevance: Excellent match (0.83)
Answer: Although returns for opened laptops are not allowed, the agent offered troubleshooting and provided manufacturer's warranty assistance, which led to the customer getting the laptop repaired or replaced by the manufacturer.
2. Returns after the specified time period in the seller's Returns Policy
Area: Cancellations and returns · Category: Replacement and Return Process · Product: Speaker
Complexity: medium · Relevance: Excellent match (0.82)
Ask about warranty, returns, compatibility, shipping, or any product support query...
Ctrl+Enter to send


How do I claim warranty for a defective product?
Can I cancel my order after it has shipped?
My product is not compatible with my device
How long does a refund take to process?
Product damaged during delivery, what to do?
How to change my delivery address after ordering?
Payment failed but amount was deducted
TOPICS COVERED
Returns & Refunds
Warranty Claims
Order Cancellation
Shipping & Delivery
Compatibility
Payment Issues
Account Access
Product Specs
customer's 2-month-old phone with battery issues.
4. Applicable Return fee
Area: Cancellations and returns · Category: Return Checks and Fees · Product: Computer Monitor
Complexity: medium · Relevance: Excellent match (0.82)
Answer: Initiate a return through the website, securely pack the product, ship it back, and wait for the refund once the product is inspected.
5. Finding seller's returns policy
Area: Order · Category: Product Information and Tags · Product: Electric Kettle
Complexity: high · Relevance: Excellent match (0.82)
Answer: The return policy for the electric kettle is 30 days from the date of delivery. The product should be in new and unused condition, and the original packaging should be intact. The return process can be initiated from the BrownBox account or by contacting customer support.
SOURCES:
Products not eligible for returns — La...
Returns after the specified time perio...
Returns after the specified time perio...
Applicable Return fee — Computer M...
Finding seller's returns policy — Electr...
Ask about warranty, returns, compatibility, shipping, or any product support query...
Ctrl+Enter to send



Benchmark Results

The system demonstrates reliable performance with low response times for product- related queries and efficient execution of the semantic retrieval pipeline.
The integration of **Sentence Transformers embeddings** with **HNSW indexing** ensures fast and accurate retrieval of relevant document chunks. The retrieval process operates efficiently using approximate nearest neighbor search, providing results in near real-time. The system avoids hallucination by relying entirely on retrieved knowledge base content instead of generating responses from external LLMs. This ensures consistent and stable outputs across different query scenarios.
Overall, the system performs efficiently under regular usage conditions, ensuring **accuracy, scalability, and reliability** for real-world customer support applications.

ADVANTAGES & LIMITATIONS

Advantages

Semantic Search Accuracy
- Uses embedding-based similarity search instead of keyword matching
- Captures user intent effectively and improves retrieval relevance

No Hallucination
- Responses are generated only from retrieved documents
- Ensures factual correctness and reliability

Fast Retrieval using HNSW
- Uses HNSW indexing for efficient nearest neighbour search
- Provides low-latency responses (O(log n) complexity)

Natural Language Interaction
- Users can ask queries in simple English
- No need for structured or predefined inputs

Modular Architecture
- Separates frontend, backend, and RAG pipeline
- Easy to maintain and extend

Scalable Design
- Supports large datasets with efficient indexing
- Can be expanded with additional knowledge sources

Limitations

Limited Knowledge Base
- System depends on available dataset
- Cannot answer queries outside the dataset

No Generative Capability
- Does not generate new responses
- Limited to extractive answers from existing data

Static Data
- Does not include real-time updates
- Requires re-indexing when data changes

Context Limitation
- Limited support for multi-turn conversations
- Each query is processed independently

CPU-Based Processing
- Embedding generation may be slower for large datasets
- No GPU acceleration by default

FUTURE ENHANCEMENTS

Scalability Improvements
- Optimize indexing for very large datasets
- Introduce distributed vector storage systems
- Implement caching for repeated queries
- Improve retrieval speed for high concurrent users

Feature Expansion
- Add support for multi-turn conversational context
- Integrate generative LLM for hybrid RAG (retrieval + generation)

- Provide summarised and contextual responses
- Enable filtering based on product categories

Real-Time Data Integration
- Integrate live product databases or APIs
- Automatically update knowledge base
- Enable dynamic response generation
- Improve relevance with real-time information

Platform Expansion
- Develop mobile-friendly UI
- Convert system into a chatbot for platforms (WhatsApp, Telegram)
- Improve UI/UX with modern frameworks
- Enable cross-platform accessibility

Automation & Intelligence
- Add query classification and intent detection
- Implement personalised recommendations
- Improve ranking of retrieved results
- Enable feedback-based learning

Conclusion

The RetailMind AI system successfully demonstrates how semantic search and Retrieval-Augmented Generation (RAG) can be used to build an accurate and reliable product support assistant.

By combining **Sentence Transformers embeddings** with **HNSW-based vector search**, the system overcomes the limitations of traditional keyword-based search systems. It ensures that user queries are understood based on meaning rather than exact wording.

Unlike generative AI systems, this approach eliminates hallucination by grounding all responses in actual knowledge base data. This significantly improves trust, accuracy, and usability.

The modular architecture built using **Flask, embedding models, and vector indexing** provides a scalable and efficient solution that can be easily extended with additional datasets and features.

Overall, this project presents a practical implementation of modern AI techniques for real-world applications, with strong potential for future enhancements such as hybrid RAG systems, real-time data integration, and conversational intelligence.

APPENDIX

Source Code

The complete source code of the project is organised into modular Python components, including backend logic, RAG pipeline, and data processing modules.

Key files include:

- app.py → Backend API and routing
- rag_pipeline.py → Embedding, retrieval, and answer generation
- data_loader.py → Data loading and preprocessing

The code follows a clean and modular architecture, ensuring readability, maintainability, and scalability.

Configuration Files

The system does not require external API configuration files such as .env, as it does not depend on external LLM services.
Configuration is handled internally within Python files, including:

- Embedding model settings
- HNSW parameters
- Runtime configurations

The requirements.txt file is used for dependency management.

Dataset Details

The system uses a publicly available dataset:

● **HuggingFace Dataset:** rjac/e-commerce-customer-support-qa
This dataset contains:

- Customer queries
- Product-related issues
- Corresponding support answers

The data is preprocessed and converted into structured chunks for semantic retrieval.

API Documentation

The project provides RESTful API endpoints for handling user queries and system operations:
● /ask → Accepts user query and returns response
● /health → Checks system status
● /index-status → Displays index readiness
Each API follows a structured JSON request-response format, ensuring smooth frontend-backend communication.

GitHub Repository Link

https://github.com/Rishika92/RetailMind-AI-Intelligent-Product-Knowledge-Engine